

Digital Clock

Madhur and Nikhil

August 2022

0.1 Objective

The main objective of this project is to create a working digital clock that displays time in HH:MM:SS format.

Additional upgrades and improvements are encouraged.

0.2 Progress Report

0.2.1 Day 0:13 Aug

The project was announced and teammate selection was introduced. At the end of the day, we decided to come together as a team.

0.2.2 Day 1: 14 Aug

We started by procuring the required components from the core labs and started tinkering to understand the components. We found that the displays were not going to work with the 7447 decoder. The 7447 initially provided was faulty, resulting in an injury.

0.2.3 Day 2: 15 Aug

The seniors informed us yesterday that a session will be held today. So, we started the installation of arduino IDE.

On ubuntu, the appimage was installed but when trying to run it, nothing was happening. We tried using windows but the IDE was becoming unresponsive after opening it.

We tried to use the cloud based platform but it was not uploading the sketch into the board.

The issue was ubuntu's new AppArmor user namespace creation restriction which prevents appimages from running. We found the code that removes it and ran it.

A reboot later, Arduino IDE is opening on ubuntu.

Since we had it running on one OS, we did not care to fix the windows issue.

Now, when we tried to upload the code, the upload always failed silently (without any error displayed). This is the result of ubuntu denying users read and write capabilities to external devices. We obtained the required code for granting permissions.

Since the origins of the code were not so trustworthy, we implemented it on only one pc. Until now, the code seems to be working. Then, we ran a blink program and confirmed it. After attending the session (which got dragged until 7 in the evening), we called it a day.

0.2.4 Day 3: 16 Aug

The Sunday morning consisted of us trying to figure out the common cathode display worked. We finally wrote a program that caused the numbers 0 to 9 to be displayed on a single display in a loop. It was afternoon by the time the program finally worked.

The evening was spent tinkering with various elements and trying to understand the code (A doubt was beginning to form that the 16 wires would not be sufficient).

0.2.5 Day 5: 17 Aug

We went to the core labs and replaced the displays and the 7447. We also brought 4 more wires, thinking we can get the rest when needed. We decided to wire the project up in tinkercad since we were unsure if the reference image was reliable (The true scale of the wires hit us). So, we spent 2 hours painfully recreating every element of the circuit only for it to show that the decoder will fry itself due to excess current. We were disappointed and ended the day.

0.2.6 Day 6: 18 Aug

The group message (about the 80 wires) confirmed our hypothesis and we got them from the lab. We studied the code of the project (linked to github) and got an idea about what it is trying to accomplish. We decided to use the columns that are normally reserved for power and ground potentials for sending simultaneous output to all displays.

(There are 8 (4+4) of them, we used 7 for the displays and the last one for the common ground of the buttons.)

Then we wired them all up in a 4hr wiring and debugging session. At the end, we discovered that there was a common cathode display in the 6 common anode displays given to us. We decided to fix it up later and proceeded with the wiring. By the end of the day, the wiring was mostly done.

0.2.7 Day 7: 19 Aug

The remaining wiring was done and we changed the code to work with our wiring and ports. By the end of the day, a clock (Albeit a barely functioning one) was created.

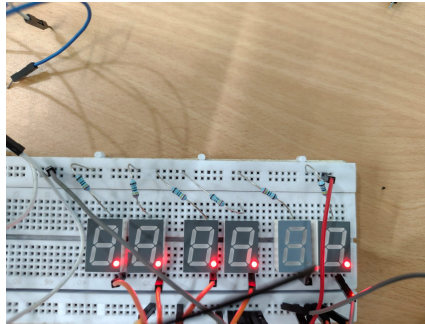


Figure 1: The common cathode display can be seen

0.2.8 Day 8: 20 Aug

We encountered a "Post-Fade" problem where the number of the previous display would faintly appear on the succeeding display. The problem was that the display was being reset after the new number was displayed, instead of before. We fixed the relevant part of the code and the problem was gone. With the core part done, we decided to start extras.

0.2.9 Day 9: 21 Aug

We finally exchanged the common cathode display for the correct one and started working on 2 things:

1. A stopwatch feature
2. Making the clock run on the frequency of the PWM output instead of `millis()`

We decided to not focus on the aesthetics as we felt the current look was good enough. The stopwatch code ended up not working(after 3 hrs of working on it). The day was ended.

0.2.10 Day 10: 22 Aug

After debugging the code(3 hrs more), we finally completed the stopwatch feature. Work commenced on using the PWM frequency to run the clock. The day ended with the PWM code not working.

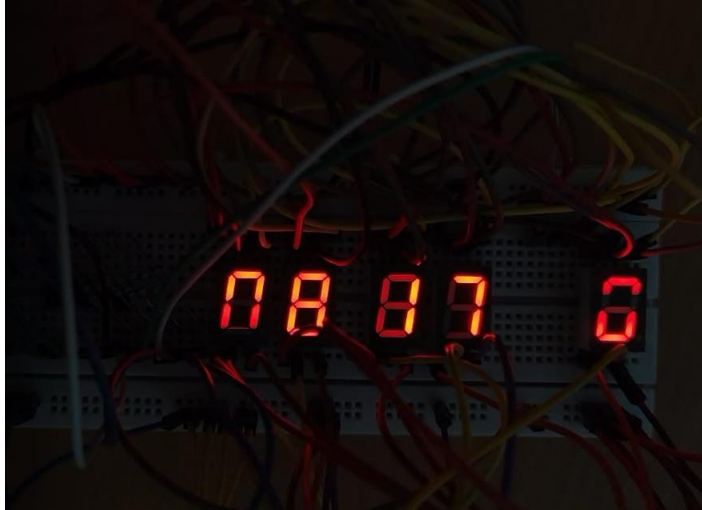


Figure 2: The impression of the previous number is on the next display

0.2.11 Day 11: 23 Aug

After fixing the code, the feature now works. For protection against the elements, we decided to add a case to the circuit.

0.2.12 Day 12: 24 Aug

We had a thought to improve our digital clock by adding a feature that whenever we connect our mobile to the Arduino the real time should be displayed in our clock, but after many trials we got a conclusion that it's not working out.

0.2.13 Day 13: 25 Aug

Since the mobile syncing was not working out, we decided to update the time during compiling. Our idea had worked but since the upload time and compile time was different there was a delay of few seconds in our clock

0.2.14 Day 14: 26 Aug

We have attended the session conducted by TA'S and finding the solution for the time delay issue. The clock started malfunctioning. The issues are:

1. When input was 0000 the output was 8888 if the number segments was 6 or more.
2. When input was 0001, the output (on display) was 3 regardless of number of segments.
3. When input was 1000, the output was 8(faint) and c(bright).
4. When input was 1001, the output was c(bright) when 6 or more segments were connected. Else, it was 0.

The rest of the day was spent removing the connections to pinpoint the issue.

0.2.15 Day 15: 27 Aug

After almost entirely disassembling the circuit, we realised that the 7447 had a faulty ground wire, which caused the issue. We changed the wire and rewired it again.

We also timed the compiling time on a laptop for 10 attempts and realised it was taking ≈ 9 seconds for compiling. So, we manually added those 9 seconds.

We also decided to time the PWM-based clock with a millis() based clock to find out which one has a higher error rate. Both had approximately same error of $\approx -10\text{sec/hr}$.

The cardboard enclosure idea was dropped because there was no way for us to provide button accessibility if the clock sat in a box.

Here is an image of the final clock

Continued in next page

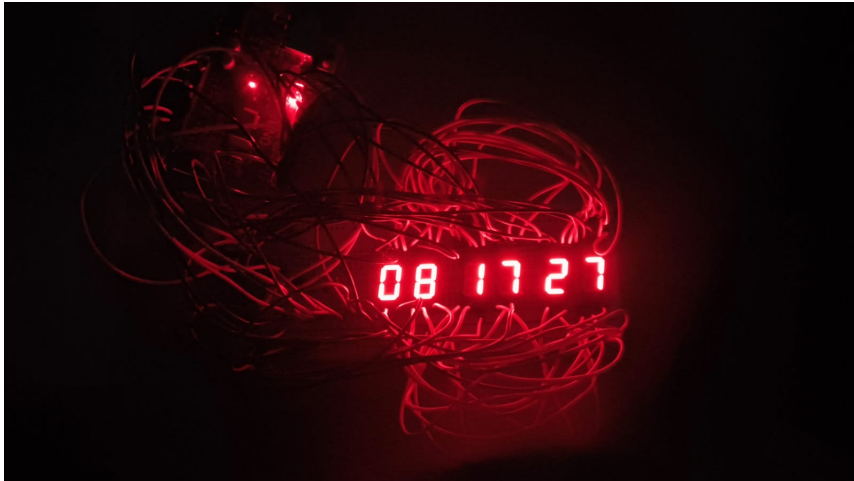


Figure 3: The Final working clock

The code for this project has been shown below:

```
// THIS CODE IS THE PWM VERSION
#define FREQPIN 2
#define PWM_OUTPUT 5
//-----The time syncer-----
char* tome=__TIME__;
int c0 =tome[0];
int c1=tome[1];
int c3= tome[3];
int c4=tome[4];
int c6=tome[6];
int c7=tome[7];

// ----- Shared 7447 Inputs -----
const int PIN_A = A2, PIN_B = A5, PIN_C = A4, PIN_D = A3;
```

```
//// Time thresholds (in milliseconds)
const unsigned long DEBOUNCE_TIME = 50;
const unsigned long LONG_PRESS_TIME = 1000;

// State variables
int lastButtonState = HIGH;
unsigned long pressedTime = 0;
unsigned long releasedTime = 0;
bool isPressing = false;
bool isLongDetected = false;

// ----- Enable pins for multiplexing -----
const int EN[] = {13,11,3,4,12,6}; // sec1, sec10, min1, min10, hr1, hr10
// ----- Buttons -----
const int PAUSE_BTN = 8;
const int NEXT_BTN = 7;
const int INC_BTN = 9;
const int DEC_BTN = 10;

bool paused = false;
bool stwatch = false;
bool strun = false;
int laststresetState=HIGH;
int lastPauseState = HIGH;
int lastNextState = HIGH;
int lastIncState = HIGH;
int lastDecState = HIGH;
int laststrunState = HIGH;

// Digit selection when paused
int selectedDigit = 0; // 0..5 → sec1..hr10
unsigned long lastBlink = 0;
bool blinkOn = true;
```

```
// ----- Current States -----  
// Seconds  
int W1=0,X1=0,Y1=0,Z1=1;  
int W2=1,X2=0,Y2=1,Z2=0;  
// Minutes  
int W3=1,X3=0,Y3=0,Z3=1;  
int W4=1,X4=0,Y4=1;  
// Hours  
int W5=1,X5=1,Y5=0,Z5=0;  
int W6=0,X6=1,Y6=0;  
  
//Seconds 1/100  
int w1=0, x1=0, y1=0, z1=0;  
//Seconds 1/10  
int w2=0, x2=0, y2=0, z2=0;  
//Seconds 1  
int w3=0, x3=0, y3=0, z3=0;  
//Seconds 10  
int w4=0, x4=0, y4=0, z4=0;  
//Minutes 1  
int w5=0, x5=0, y5=0, z5=0;  
//Minutes 10  
int w6=0, x6=0, y6=0, z6=0;  
  
// Time variables  
unsigned long lastSecUpdate=0;  
unsigned long lastStwUpdate=0;  
volatile uint32_t sec_ISR;  
  
//-----ISR FUNC-----  
void freqCounter()  
{  
    sec_ISR++;  
}  
int asciimaker(int num){
```

```
    return (num-48);
}
//Decimal to binary converter
int binnum[4]={0,0,0,0};
void binconverter(int num){

    int i=0;
    if (num==0){
        Serial.println(0);
        return;
    }
    while (num>0){
        binnum[i]=num%2;
        num=num/2;
        i++;
    }

    //for(int j=3; j>=0;j--){
    //Serial.print(binnum[j]);    ///IMPORTANT NOTE: THE ORDER IS REVERSED!!!!
    //}

    Serial.println();
}
void clean(){
    for (int i=0;i<4;i++){
        binnum[i]=0;
    }
}

// ----- Setup -----
void setup(){
    c0=c0-48;
    c1=c1-48;
    c3=c3-48;///
```

```
c4=c4-48;///  
c6=c6-48;  
c7=c7-48;///  
if ((c7+9)>10){  
    if(c6==5){  
        c6=0;///  
    }  
    else{c6+=1;}  
}  
c7+=9;  
Serial.begin(9600);  
Serial.println();  
delay(1000);  
pinMode(PIN_A,OUTPUT); pinMode(PIN_B,OUTPUT);  
pinMode(PIN_C,OUTPUT); pinMode(PIN_D,OUTPUT);  
for(int i=0;i<6;i++){ pinMode(EN[i],OUTPUT); digitalWrite(EN[i],LOW); }  
pinMode(PAUSE_BTN,INPUT_PULLUP);  
pinMode(NEXT_BTN,INPUT_PULLUP);  
pinMode(INC_BTN,INPUT_PULLUP);  
pinMode(DEC_BTN,INPUT_PULLUP);  
attachInterrupt(digitalPinToInterrupt(2),freqCounter,RISING);  
analogWrite(PWM_OUTPUT,127);  
  
clean();  
binconverter(c7);  
W1=binnum[0];  
X1=binnum[1];  
Y1=binnum[2];  
Z1=binnum[3];  
clean();  
binconverter(c6);  
W2=binnum[0];  
X2=binnum[1];  
Y2=binnum[2];  
clean();  
binconverter(c4);
```

```

W3=binnum[0];
X3=binnum[1];
Y3=binnum[2];
Z3=binnum[3];
clean();
binconverter(c3);
W4=binnum[0];
X4=binnum[1];
Y4=binnum[2];

clean();
binconverter(c1);
W5=binnum[0];
X5=binnum[1];
Y5=binnum[2];
Z5=binnum[3];
clean();
binconverter(c0);
W6=binnum[0];
X6=binnum[1];
Y6=binnum[2];

}

// ----- Increment Helper -----
void incrementDigit(int d){
    int A,B,C,D;
    switch(d){
        case 0: { // sec ones 0{9
            A = !W1;
            B = (W1 && !X1 && !Z1)||(!W1 && X1);
            C = (!X1 && Y1)||(!W1 && Y1)||(W1 && X1 && !Y1);
            D = (!W1 && Z1)|| (W1 && X1 && Y1);
            W1=A; X1=B; Y1=C; Z1=D;
        } break;
        case 1: { // sec tens 0{5

```

```

    A = !W2;
    B = (W2 && !X2 && !Y2)||(!W2 && X2);
    C = (W2 && X2)||(!W2 && !X2 && Y2);
    W2=A; X2=B; Y2=C;
} break;
case 2: { // min ones 0{9
    A = !W3;
    B = (W3 && !X3 && !Z3)||(!W3 && X3);
    C = (!X3 && Y3)||(!W3 && Y3)|| (W3 && X3 && !Y3);
    D = (!W3 && Z3)|| (W3 && X3 && Y3);
    W3=A; X3=B; Y3=C; Z3=D;
} break;
case 3: { // min tens 0{5
    A = !W4;
    B = (W4 && !X4 && !Y4)||(!W4 && X4);
    C = (W4 && X4)||(!W4 && !X4 && Y4);
    W4=A; X4=B; Y4=C;
} break;
case 4: { // hr ones
    if(X6==0){ // hr tens=0/1 → 0{9
        A = !W5;
        B = (W5 && !X5 && !Z5)||(!W5 && X5);
        C = (!X5 && Y5)||(!W5 && Y5)|| (W5 && X5 && !Y5);
        D = (!W5 && Z5)|| (W5 && X5 && Y5);
        W5=A; X5=B; Y5=C; Z5=D;
    } else { // hr tens=2 → 0{3
        A = !W5;
        B = (W5 && !X5)||(!W5 && X5);
        W5=A; X5=B; Y5=0; Z5=0;
    }
} break;
case 5: { // hr tens 0{2
    if(!(X6==0 && W6==1 && Y5==1)){
        A = !W6 && !X6;
        B = W6 && !X6;
        W6=A; X6=B; Y6=0;

```

```

    }
    } break;
}
}

// ----- Decrement Helper -----
void decrementDigit(int d){
    int A,B,C,D;
    switch(d){
        case 0: { // sec ones 0{9
            A = !W1;
            B = (!X1 && !W1 && ((!Z1 && Y1)|| (Z1 && !Y1))) || (!Z1 && W1 && X1);
            C = (!Z1 && Y1 && (X1||W1)) || (Z1 && !X1 && !W1 && !Y1);
            D = !X1 && !Y1 && ((Z1 && W1) || (!Z1 && !W1));
            W1=A; X1=B; Y1=C; Z1=D;
        } break;
        case 1: { // sec tens 0{5
            A = !W2;
            B = (Y2 && !X2 && !W2) || (!Y2 && X2 && W2);
            C = !X2 && ((Y2 && W2) || (!Y2 && !W2));
            D = 0;
            W2=A; X2=B; Y2=C;
        } break;
        case 2: { // min ones 0{9
            A = !W3;
            B = (!X3 && !W3 && ((!Z3 && Y3)|| (Z3 && !Y3))) || (!Z3 && W3 && X3);
            C = (!Z3 && Y3 && (X3||W3)) || (Z3 && !X3 && !W3 && !Y3);
            D = !X3 && !Y3 && ((Z3 && W3) || (!Z3 && !W3));
            W3=A; X3=B; Y3=C; Z3=D;
        } break;
        case 3: { // min tens 0{5
            A = !W4;
            B = (Y4 && !X4 && !W4) || (!Y4 && X4 && W4);
            C = !X4 && ((Y4 && W4) || (!Y4 && !W4));
            D = 0;
            W4=A; X4=B; Y4=C;
        }
    }
}

```

```

    } break;
    case 4: { // hr ones
        if(X6==0){ // hr tens=0/1
            A = !W5;
            B = (!X5 && !W5 && ((!Z5 && Y5)|| (Z5 && !Y5))) || (!Z5 && W5 && X5);
            C = (!Z5 && Y5 && (X5||W5)) || (Z5 && !X5 && !W5 && !Y5);
            D = !X5 && !Y5 && ((Z5 && W5) || (!Z5 && !W5));
            W5=A; X5=B; Y5=C; Z5=D;
        } else { // hr tens=2 → 0{3
            A = !W5;
            B = (X5 && W5) || (!X5 && !W5);
            W5=A; X5=B; Y5=0; Z5=0;
        }
    } break;
    case 5: { // hr tens 0{2
        if(!(X6==0 && W6==0 && Y5==1)){
            A = X6 && !W6;
            B = !X6 && !W6;
            W6=A; X6=B; Y6=0;
        }
    } break;
}

// ----- Main Loop -----
void loop(){
    if(sec_ISR>=976){

        // Time update
        if(!paused ){
            // Seconds Ones
            int A = !W1;
            int B = (W1 && !X1 && !Z1)||(!W1 && X1);
            int C = (!X1 && Y1)||(!W1 && Y1)|| (W1 && X1 && !Y1);
            int D = (!W1 && Z1)|| (W1 && X1 && Y1);
            W1=A; X1=B; Y1=C; Z1=D;

```

```

// Seconds Tens
if((W1|X1|Y1|Z1)==0){
    A = !W2;
    B = (W2 && !X2 && !Y2)||(!W2 && X2);
    C = (W2 && X2)||(!W2 && !X2 && Y2);
    W2=A; X2=B; Y2=C;
}
// Minutes Ones
if((W1|X1|Y1|Z1|W2|X2|Y2)==0){
    A = !W3;
    B = (W3 && !X3 && !Z3)||(!W3 && X3);
    C = (!X3 && Y3)||(!W3 && Y3)||(W3 && X3 && !Y3);
    D = (!W3 && Z3)||(!W3 && X3 && Y3);
    W3=A; X3=B; Y3=C; Z3=D;
// Minutes Tens
if((W3|X3|Y3|Z3)==0){
    A = !W4;
    B = (W4 && !X4 && !Y4)||(!W4 && X4);
    C = (W4 && X4)||(!W4 && !X4 && Y4);
    W4=A; X4=B; Y4=C;
}
// Hours Ones
if((W3|X3|Y3|Z3|W4|X4|Y4)==0){
    if(X6==0){
        A = !W5;
        B = (W5 && !X5 && !Z5)||(!W5 && X5);
        C = (!X5 && Y5)||(!W5 && Y5)||(W5 && X5 && !Y5);
        D = (!W5 && Z5)||(!W5 && X5 && Y5);
        W5=A; X5=B; Y5=C; Z5=D;
    } else {
        A = !W5;
        B = (W5 && !X5)||(!W5 && X5);
        W5=A; X5=B; Y5=0; Z5=0;
    }
// Hours Tens
if((W5|X5|Y5|Z5)==0){

```

```

        A = !W6 && !X6;
        B = W6 && !X6;
        W6=A; X6=B; Y6=0;
    }
}
}
    sec_ISR=0;
}
// Pause toggle
int pauseState = digitalRead(PAUSE_BTN);
int strunState =digitalRead(PAUSE_BTN);
//int strunState=HIGH;
if (stwatch && strunState==LOW && laststrunState==HIGH){
    //Serial.println("play");
    if(!isPressing){
        strun=!strun;
    }
    isPressing= true;
    //strun=!strun;
}
else if (laststrunState=HIGH && strunState==LOW && stwatch){
    isPressing= false;
}
laststrunState==strunState;

if(pauseState==LOW && lastPauseState==HIGH && !stwatch){
    paused=!paused;
    if(paused) selectedDigit=0;
    delay(50);
}
lastPauseState=pauseState;
//int stresetState=digitalRead(INC_BTN);
int incState=digitalRead(INC_BTN);
if(incState==LOW && lastIncState==HIGH && stwatch && !strun){

```

```
w1=0;
w2=0;
w3=0;
w4=0;
w5=0;
w6=0;
x1=0;
x2=0;
x3=0;
x4=0;
x5=0;
x6=0;
y1=0;
y2=0;
y3=0;
y4=0;
y5=0;
y6=0;
z1=0;
z2=0;
z3=0;
z4=0;
z5=0;
z6=0;
}
```

```
// Next cursor
```

```
int nextState=digitalRead(NEXT_BTN);
```

```
//nextState = digitalRead(NEXT_BTN);
```

```
if (lastNextState == HIGH && nextState == LOW) {
```

```
  //Serial.println("IAMHERE");
```

```
  pressedTime = millis();
```

```
    isPressing = true;
    isLongDetected = false;
}

else if (lastNextState == LOW && nextState == HIGH) {
    releasedTime = millis();
    isPressing = false;

    long pressDuration = releasedTime - pressedTime;

    if (pressDuration > DEBOUNCE_TIME && pressDuration < LONG_PRESS_TIME) {
        if(pressed && nextState==HIGH && lastNextState==LOW){
            selectedDigit=(selectedDigit+1)%6;
            delay(50);
        }
    }
}

if (isPressing == true && isLongDetected == false) {
    long pressDuration = millis() - pressedTime;

    if (pressDuration >= LONG_PRESS_TIME) {
        stwatch=!stwatch;
        isLongDetected = true;
    }
}

lastNextState = nextState;
/*if (stwatch){
    Serial.println("stwatch is true");
}*/
if (stwatch){
    for(int d =0;d<6;d++){
```

```
//Serial.println("IAM HERE");
switch(d){
    case 0: showDigit(w1,x1,y1,z1,EN[0]); break;
    case 1: showDigit(w2,x2,y2,z2,EN[1]); break;
    case 2: showDigit(w3,x3,y3,z3,EN[2]); break;
    case 3: showDigit(w4,x4,y4,0,EN[3]); break;
    case 4: showDigit(w5,x5,y5,z5,EN[4]); break;
    case 5: showDigit(w6,x6,y6,0,EN[5]); break;
}
delay(1);
}
}
// Increment

if(paused && incState==LOW && lastIncState==HIGH){
    incrementDigit(selectedDigit);
    delay(50);
}
lastIncState=incState;

// Decrement
int decState=digitalRead(DEC_BTN);
if(paused && decState==LOW && lastDecState==HIGH){
    decrementDigit(selectedDigit);
    delay(50);
}
lastDecState=decState;

// Blink
if(paused && millis()-lastBlink>=500){
    blinkOn=!blinkOn;
    lastBlink=millis();
}

// Display
for(int d=0; d<6; d++){
```

```

bool showThis=true;
if(paused && d==selectedDigit && !blinkOn) showThis=false;
if(showThis && !stwatch){
    switch(d){
        case 0: showDigit(W1,X1,Y1,Z1,EN[0]); break;
        case 1: showDigit(W2,X2,Y2,0,EN[1]); break;
        case 2: showDigit(W3,X3,Y3,Z3,EN[2]); break;
        case 3: showDigit(W4,X4,Y4,0,EN[3]); break;
        case 4: showDigit(W5,X5,Y5,Z5,EN[4]); break;
        case 5: showDigit(W6,X6,Y6,0,EN[5]); break;
    }
    delay(1);
}
}

if( strun && millis()-lastStwUpdate>=10){
    lastStwUpdate=millis();
    //Seconds 1/100
    int SA=!w1;
    int SB=(w1 && !x1 && !z1)||(!w1 && x1);
    int SC=!x1 && y1)||(!w1 && y1)||(w1 && x1 && !y1);
    int SD=!w1 && z1)||(w1 && x1 && y1);
    w1=SA; x1=SB; y1=SC ; z1=SD;
    // Seconds 1/10
    if ((w1|x1|y1|z1)==0){
        SA=!w2;
        SB=(w2 && !x2 && !z2)||(!w2 && x2);
        SC=!x2 && y2)||(!w2 && y2)||(w2 && x2 && !y2);
        SD=!w2 && z2)||(w2 && x2 && y2);
        w2=SA; x2=SB; y2=SC ; z2=SD;
    }
    //Seconds 1
    if((w2|x2|y2|z2|w1|x1|y1|z1)==0){
        SA=!w3;
        SB=(w3 && !x3 && !z3)||(!w3 && x3);
        SC=!x3 && y3)||(!w3 && y3)||(w3 && x3 && !y3);
    }
}

```

```

SD=(!w3 && z3)|| (w3 && x3 && y3);
w3=SA; x3=SB; y3=SC ; z3=SD;

//Seconds 10
if((w3|x3|y3|z3|w2|x2|y2|z2)==0){
    SA = !w4;
    SB = (w4 && !x4 && !y4)||(!w4 && x4);
    SC = (w4 && x4)||(!w4 && !x4 && y4);
    w4=SA; x4=SB; y4=SC;
    if ((w4|x4|y4|w3|x3|y3|z3)==0){
        SA=!w5;
        SB=(w5 && !x5 && !z5)||(!w5 && x5);
        SC=(!x5 && y5)||(!w5 && y5)|| (w5 && x5 && !y5);
        SD=(!w5 && z5)|| (w5 && x5 && y5);
        w5=SA; x5=SB; y5=SC ; z5=SD;

        if ((w5|x5|y5|z5|w4|x4|y4)==0){
            SA = !w6;
            SB = (w6 && !x6 && !y6)||(!w6 && x6);
            SC = (w6 && x6)||(!w6 && !x6 && y6);
            w6=SA; x6=SB; y6=SC;
        }
    }
}

}

}

}

}

// ----- Display Helper -----
void showDigit(int A,int B,int C,int D,int ENpin){
    for(int i=0;i<6;i++) digitalWrite(EN[i],LOW);
    digitalWrite(PIN_A,A); digitalWrite(PIN_B,B);
    digitalWrite(PIN_C,C); digitalWrite(PIN_D,D);
    digitalWrite(ENpin,HIGH);
}

```

```
}
```